

A new disk-based technique for solving the largeness problem of stochastic modeling formalisms

Samir M. Koriem ^{a,*}, W.S. El-Kilani ^b

^a Department of Systems & Computers Engineering, Faculty of Engineering, El-Azhar University, Cairo, Egypt

^b Department of Information Technology, Faculty of Computers & Information, Menoufia University, Shabeen El-Koum, Egypt

Received in revised form 5 November 2002; accepted 12 April 2003

Available online 5 December 2003

Abstract

Stochastic modeling formalisms such as stochastic Petri nets, generalized stochastic Petri nets, and stochastic reward nets can be used to model and evaluate the dynamic behavior of realistic computer systems. Once we translate the stochastic system model to the underlying corresponding Markov Chain (MC), the developed MC grows wildly to several hundred thousands states. This problem is known as the largeness problem. To tolerate the largeness problem of Markov models, several iterative and direct methods have been proposed in the literature. Although the iterative methods provide a feasible solution for most realistic systems, a major problem appears when these methods fail to reach a solution. Unfortunately, the direct method represents an undesirable numerical technique for tolerating large matrices due to the fill-in problem. In order to solve such problem, in this paper, we develop a disk-based segmentation (DBS) technique based on modifying the Gauss Elimination (GE) technique. The proposed technique has the capability of solving the consequences of the fill-in problem without making assumptions about the underlying structure of the Markov processes of the developed model. The DBS technique splits the matrix into a number of vertical segments and uses the hard disk to store these segments. Using the DBS technique, we can greatly reduce the memory required as compared to that of the GE technique. To minimize the increase in the solution time due to the disk accessing processes, the DBS utilizes a clever management technique for such processes. The effectiveness of the DBS technique has been demonstrated by applying it to a realistic model for the Kanban manufacturing system.

© 2003 Elsevier Inc. All rights reserved.

Keywords: Stochastic reward nets; Largeness tolerance methods; Direct methods; Markov chain; Disk-based segmentation; Gauss elimination

1. Introduction

Rapid advances in technology have resulted in the proliferation of complex computer systems that are used in different applications, ranging from spacecraft flight control to information and financial services. Modeling and evaluating mechanisms provide a good methodology for examining the behavior of these systems from the design stage to the implementation and final deployment. Recently, the use of Continuous Time Markov Chains (CTMCs) has proven to be a very useful modeling formalism for evaluating these systems

(Mahevas and Rubino, 2001; Nicola et al., 2001). Many problems have been encountered in the use of Markov models such as *stiffness* and *largeness* (Abdallah et al., 2000; Mahevas and Rubino, 2001).

Stiffness is an undesirable characteristic of many practical Markov models. Stiffness adversely affects the computational efficiency of the numerical solution technique (Abdallah et al., 2000). The stiffness problem arises if the model solution has rates that differ widely. Moreover, since most Markov models of real systems are very large, the actual model of the desired system is usually specified using a high-level description such as Stochastic Reward Nets (SRNs) (Ramani et al., 2000). The SRN technique has the capability of modeling the dynamic behavior of large and complex systems in a compact way. Once we translate the SRN system model to the underlying corresponding CTMC, the developed

* Corresponding author. Address: College of Technology, P.O. Box: 1604, Abha, Saudi Arabia.

E-mail address: samirkoriem@yahoo.com (S.M. Koriem).

CTMC grows wildly to several hundred thousands states. This problem is known as the *largeness problem* (Goseva-Popstojanova and Trivedi, 2000).

The largeness problem can be overcome by the use of the largeness avoidance or largeness tolerance technique (Goseva-Popstojanova and Trivedi, 2000). The *largeness avoidance* approach needs to discover the structure of the CTMC under study to facilitate its solution. Examples of such technique are the stochastic process algebras, matrix-geometric stochastic Petri nets, product-form queuing networks, and product-form stochastic process algebras (Gribaudo et al., 2001; Haverkort et al., 1997; Hermanns et al., 2002). These techniques have the capability of exploiting the structure of the CTMC under study to obtain an efficient CTMC generation. Subsequently, the solution of the CTMC becomes possible.

On the other hand, the *largeness tolerance* approach employs the sparse storage techniques and computational procedures that do well in combination with sparsely processing solution methods. This approach gives a facility to the modeler to accommodate the desired models that are as large as possible. In practice, the concept of this approach means that the CTMC size is restricted by the size of the main memory of the system on which the model is constructed and solved. CTMCs with several hundred thousand states have been solved using this approach (Chiola et al., 1995). It should be noted that the model reduction is employed only in the case of the largeness avoidance approach. In this paper, we deal only with the largeness tolerance approach. In the literature, three important numerical solution techniques for the largeness tolerance problem have been considered: Superposed Generalized Stochastic Petri Nets (SGSPNs) (Buchholz, 2001), on-the-fly technique (Deavours and Sanders, 1998b) and disk-based method (Deavours and Sanders, 1998a).

The idea of the SGSPN technique (Buchholz, 2001) is to combine a set of originally independent GSPNs into a single superposed GSPN by synchronization of transitions. At net level, the synchronization takes place by merging the desired transitions. The resulting transitions are called synchronized transitions. This concept is closely related to the concept of stochastic automata networks (Plateau et al., 1999) and Markovian process algebras (Buchholz and Kemper, 2001). The SGSPN technique has two substantial restrictions: (i) the structure of the developed model should incorporate independent components with limited interaction between them, and (ii) the sub-models of the developed model should have approximately the same size. Therefore, the SGSPN technique has the disadvantage of requiring a special structure in the model in order to work efficiently (Buchholz and Kemper, 2001).

On-the-fly technique (Deavours and Sanders, 1998b) introduces a new iterative solution method, called a modified adaptive Gauss–Seidel. This technique elimi-

nates the need to explicitly store the matrix at all when generating the rows and columns of the state transition-rate-matrix on-the-fly. Further, this approach permits the caching of portions of the matrix. This process leads to the reduction of the solution time.

A disk-based technique (Deavours and Sanders, 1998a) uses a workstation disk to hold the transition-rate matrix of the CTMC under study, the variant of the Gauss–Seidel block as an iterative solution method, and an innovative implementation that involves the following two parallel processes. The first process retrieves portions of the transition matrix from the workstation disk. The second process repeats the computation on small portions of the matrix. Disk-based methods have the potential to greatly outperform the SGSPN method by using a large disk to allow to the holding of the state-transition-rate matrix of the Markov model being solved. Further, high performance disks are inexpensive, relative to the cost of main memories (RAMs).

It is interesting to note that largeness tolerance methods such as the SGSPN (Buchholz, 2001), the on-the-fly technique (Deavours and Sanders, 1998b) and the disk-based method (Deavours and Sanders, 1998a) utilize powerful iterative methods such as the Jacobi method, the power method, and the Gauss–Seidel method. A major problem appears when the largeness tolerance methods fail to reach a solution for the desired model or spend a long time to converge to the prescribed precision (German, 2001). To solve such problem, we can use the *direct method* as an alternative approach for the numerical solution of the Markov models. Unfortunately, the direct method is an undesirable numerical technique for tolerating *large matrices* due to the following problems (Qureshi et al., 1996; Stewart, 1994).

- The reduction of the matrix from its regular form to the upper triangular form (reduction phase) often needs to perform subtraction operations among the rows. These operations lead to the creation of many non-zero elements in positions that previously contained zeros and the fading out of the undesired elements. This kind of problem is known as the *fill-in* problem. The appearance of new elements and the fading out of undesired elements needs a continuous process of data insertion and deletion from the memory. This process needs a lot of time. Consequently, the direct method suffers from a time consumption problem.
- The subtraction operations among the rows that occur during the reduction phase also lead to a continuous change of matrix elements. This change in turn leads to a build up of rounding error in the matrix elements.

The main objective of this paper is to develop a suitable largeness tolerance methodology based on

modifying a version of the Gauss Elimination (GE) technique. The proposed technique has the capability of solving the consequences of the *fill-in* problem that usually arises in the GE procedure. Our proposed methodology modifies the GE procedure in order to be able to solve the large Markov chain matrices generated from the stochastic reward nets. We call such a technique “disk-based segmentation” (DBS) technique. The basic concept of the DBS technique depends on splitting the whole matrix into a number of vertical segments. Each vertical segment consists of a band of successive columns. Each vertical segment is acted upon by a modified version of the GE procedure in such a way that these segments are transformed and become a part of the triangular form of the original matrix. We make use of the hard disk to store the segments before and after the transformation processes and carry the records of the computation operations applied. It is interesting to note that the dependence of the proposed technique on the hard disk for data retrieval and loading gives it the name DBS method. In order to identify the merits of our proposed technique, we measure its efficiency with respect to time and space under different conditions. We have also implemented a tool to support the proposed methodology. This tool works on PCs and provides us with the following capabilities.

- Generating and solving Markov large matrices resulting from the SRN models.
- Performing both steady and transient analysis for these matrices by utilizing a number of traditional direct and iterative methods proposed in the literature (Abdallah et al., 2000; Buchholz et al., 2000; Mahevas and Rubino, 2001; Nicola et al., 2001).

The remaining parts of this paper are organized as follows. Section 2 gives a brief description for our developed EasySPN tool that is used to support the DBS technique. Section 3 illustrates the main steps of the proposed DBS technique. In Section 4, three implementation issues for the DBS technique have been analyzed. Section 5 gives the numerical results for the application processes of the DBS technique on a large matrix generated from a practical SRN system model. Section 6 concludes this paper.

2. The EasySPN tool

In this section, we give a brief description of the proposed tool that is used to support the DBS technique. The proposed tool is called the *EasySPN*. This tool has the capability of generating and solving the Markov model that is obtained from the SRN system model. The EasySPN tool is a windows-based application that runs on Microsoft (MS) Windows 2000 (or its

later versions). The capability of utilizing the powerful memory management facilities of MS Windows represents the main advantage of the proposed tool. Further, the proposed tool has been implemented in a modular fashion using the object-oriented facilities of MS Visual C++ package.

This *EasySPN* package consists of five main modules: the graphical user interface (GUI) module, the state space generator (SSG) module, the state probability solver (SPS) module, the performance measures (PM) module, and the memory management (MM) module. A block diagram of the tool is shown in Fig. 1. The GUI module manipulates four menus: net menu, firing menu, solver menu and performance menu. The net menu allows the user to enter the description of the modeled system according to the GSPN, or SRN formalisms. Using the SSG module, the firing menu permits firing of the desired net giving rise to the Reachability Set (**RS**) and the Reachability Graph (**RG**). The **RS** is the set of states resulting from firing the transitions of the SRN model. The description of the connections (in terms of the transition rates of the SRN model) between the various states of the **RS** is contained in the **RG**. Both the **RS** and **RG** are stored in two separate files in a way allowing easy access by other modules of the program. It is well known that the developed **RG** of the stochastic nets is isomorphic to an ergodic CTMC (Stewart, 1994). This CTMC is governed by the following Kolmogorov differential equation (Qureshi et al., 1996).

$$d\pi(\theta)/d\theta = \pi(\theta)Q; \quad \theta \geq 0 \quad (2.1)$$

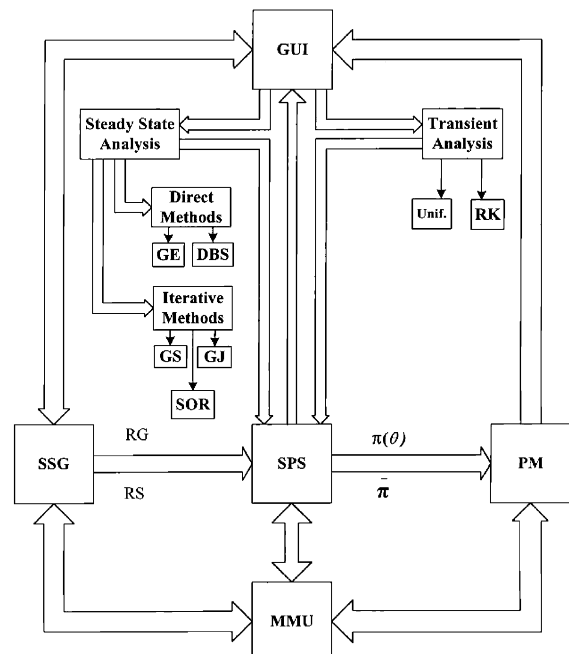


Fig. 1. Block diagram of the *EasySPN* tool.

with

$$\sum_{i=0}^{N-1} \pi_i(\theta) = 1 \quad \text{for all } \theta \geq 0 \quad (2.2)$$

where N is the number of the **RS** states, $\pi \equiv \{\pi_i; 0 \leq i \leq N-1\}$, and $\pi_i(\theta)$ is the probability of being in a state S_i at time θ . The matrix $\mathbf{Q} \equiv \{q_{ij}\}_{N \times N}$ is the infinitesimal generator or transition rate matrix of the CTMC under study. Each element of the matrix $\mathbf{Q}$ is related to the infinitesimal transition rate λ_{ij} from a state S_i to a state S_j as follows.

$$q_{ij} = \sum_{t \in T: S_i \xrightarrow{t} S_j} \lambda_{it}; \quad 0 \leq i \leq N-1 \quad (2.3)$$

With the constraints:

$$q_{ii} = - \sum_{\substack{j=0 \\ i \neq j}}^{N-1} q_{ij} \quad (2.4)$$

The SPS module computes $\pi(\theta)$ as well as the steady state distribution $\bar{\pi}$ using Eqs. (2.1) and (2.2), and the following equation.

$$\bar{\pi} \mathbf{Q} = 0, \quad \sum_{i=0}^{N-1} \bar{\pi}_i = 1 \quad (2.5)$$

We have supported the SPS module with several well-known algorithms for steady state analysis such as Gauss Seidal, Gauss Jordan and successive over-relaxation. Also, several algorithms for transient analysis such as Runge Kuta, and uniformization have supported the SPS module. Depending on the nature of the net (i.e. values of its $\mathbf{Q}$ elements), the user may choose any of the previously mentioned techniques using items of the solver menu. Using $\pi(\theta)$ and $\bar{\pi}$, the user can compute various performance, reliability and availability estimates of the modeled system by programming the PM modules. The execution of PM programs is activated using items of the performance menu. Finally, the MM module is responsible for allocating enough memory blocks for the different modules of the package. The MM module also allows clever retrieval and storage of data through these memory blocks.

Although several packages have been implemented (under Sun OS and Linux) to deal with SPNs such as: SPNP (Hirel et al., 2000), GreatGSPN (Chiola et al., 1995) and DSPNExpress (Lindemann, 1995), our package has the ability to work with MS Windows. This advantage enables our package to interact with other MS packages. Another advantage of our package is its ability to manage the full memory of the PC. Hence, our package performs on the PC as other packages perform on other platforms (e.g. Sun platforms). We will not go further in the description of our package, since this is not the objective of this paper.

3. The disk-based segmentation technique

In this section, we propose the DBS technique for solving the fill-in problem. This technique depends on modifying the classical GE method in which the whole matrix is reduced to its upper triangular form by eliminating non-zero elements under the matrix diagonal. The basic concept of the DBS technique depends on splitting the whole matrix into a number of vertical segments K . Each vertical segment consists of a band of successive columns. To obtain the upper triangular form of the matrix, the DBS technique transforms each vertical segment (denoted by VS_l ; $1 \leq l \leq K$) to the *Segmental Upper Triangular* (SUT) form. The path of the original matrix diagonal through the segment VS_l determines the SUT form of this segment. Since the matrix diagonal traverses each $\text{VS}_l \in \mathbf{VS} \equiv \{\text{VS}_l; 1 \leq l \leq K\}$ in a different position, each VS_l will have its own SUT form. The SUT form of VS_l can be defined as follows.

$$\text{VS}_l(x, y) = \begin{cases} = 0 & \text{If } (\text{Col}_{bl} \leq x \leq \text{Col}_{el}), \quad (\text{Col}_{bl} \leq y \leq x-1) \\ = 0 & \text{If } (x > \text{Col}_{el}) \\ = \text{any value} & \text{otherwise} \end{cases}$$

where $\text{VS}_l(x, y)$: represents the value of the VS_l element at row x and column y in the matrix $\mathbf{Q}^T$; Col_{bl} : represents the first column in the segment VS_l ; Col_{el} : represents the last column in the segment VS_l .

The SUT form of the segment VS_l can be obtained by performing a *reduction analysis* (RA) on this segment. The RA of VS_l means the elimination processes of non-zero elements under the path of the matrix diagonal through VS_l . Applying subtraction operations between the rows of these non-zero elements and the corresponding diagonal rows after scaling the diagonal rows can perform these elimination processes. By diagonal rows, we mean the rows of the vertical segment intersecting the path of the diagonal. Computational operations performed during the RA of VS_l are stored on the *hard disk* (denoted by HD). We call these operations, the *history list* of VS_l (denoted by HL_l).

Moreover, before applying the RA of vertical segment VS_l , we must first apply on VS_l the computational operations (division and subtraction operations) that have been performed during the RA of previous vertical segments, i.e. HL_i ; $1 \leq i \leq l-1$. We call this step, the *history analysis* of VS_l (denoted by HA_l). Accordingly, the z th history analysis of VS_l means applying the computational operations (division and subtraction operations) that have been performed during the RA of $\text{VS}_z \in \mathbf{VS}$; $1 \leq i \leq l-1$ to the current segment VS_l . The reason for this step can be explained as follows. Each vertical segment contains segments of all rows (e.g. row $_i$) of the original matrix $\mathbf{Q}^T$ (transpose of $\mathbf{Q}$). Hence, the compu-

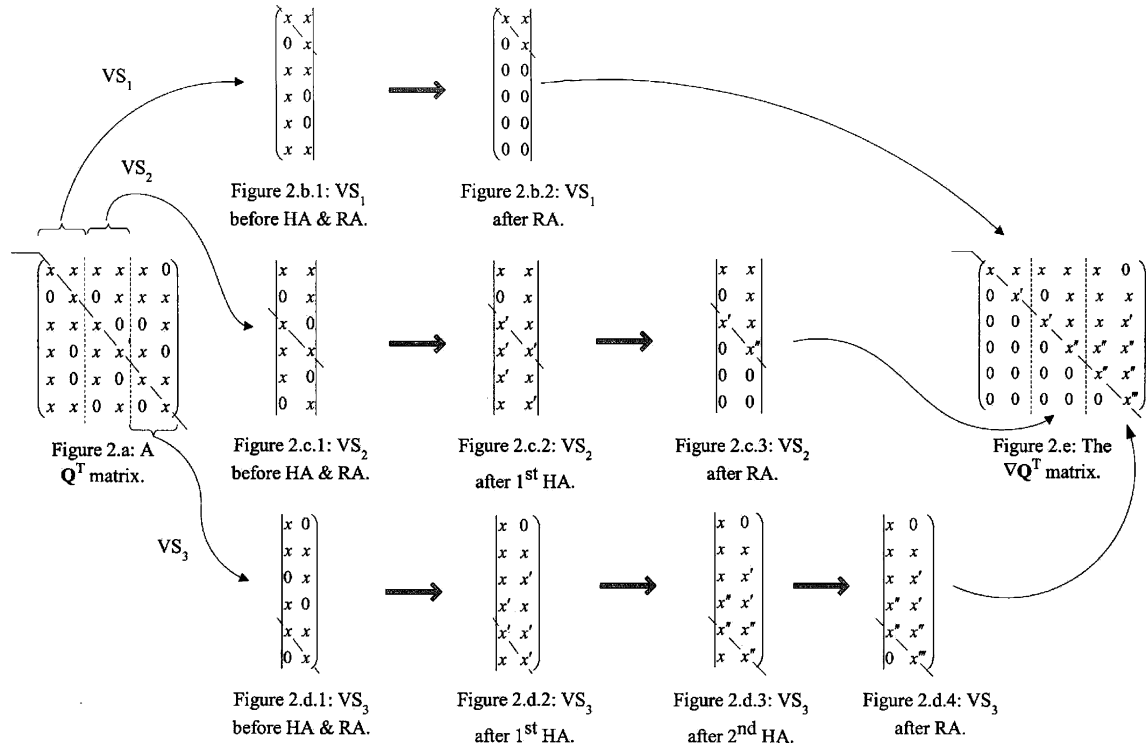


Fig. 2. The DBS technique.

tational operations performed on one segment of row_{*i*} in one vertical segment VS_j should be applied also to the other segments of row_{*i*} in the vertical segments VS_l ; $l > j$.

Fig. 2 illustrates the steps of the DBS technique, where the DBS technique is applied on a matrix split into three vertical segments. The DBS scheme can be summarized in the following steps.

Algorithm: The DBS technique steps.

- Step 1.** Develop the full matrix Q^T ($N \times N$) from the RG of the desired model (see Fig. 2a).
- Step 2.** Partition the matrix Q^T into a number K of vertical segments (Fig. 2b.1, c.1, and d.1). The set of vertical segments VS is defined as $VS = \{VS_1, VS_2, \dots, VS_K\}$. This set gives the whole matrix Q^T . Each vertical segment VS_l consists of N rows $\times$ M columns (Fig. 2b.1).
- Step 3.** Store each VS_l in a file on the HD. The storage processes of the vertical segments have been implemented in a suitable manner so that the overhead of retrieving them from the HD is minimized.
- Step 4.** Perform the following operations on the segments $VS_1, VS_2, \dots, VS_K$ to transform the full matrix Q^T into its triangular form ∇Q^T .

For $l = 1$ To K do

- (a) Retrieve VS_l from the HD to the RAM. This RAM represents the available memory that is used for applying our DBS technique on the

desired model. The determination of RAM can be done according to the resources of the platform that is used to run the algorithm.

- (b) Use the following loop to perform the HA_l .

For $z = 1$ To $l - 1$ do

Perform the z th history analysis of VS_l , or apply the computational operations contained in HL_z to VS_l . Fig. 2c.2 shows the application of the computational operations contained in HL_1 to VS_2 .

- (c) Use the RA to transform VS_l into its SUT form. Fig. 2c.3 shows the SUT form of VS_2 after the application of its RA.
- (d) Save a list of all operations that have been done during the RA of VS_l to HL_l .
- (e) Store the final SUT form of VS_l on the HD in a separate file.

- Step 5.** Calculate the steady state probabilities $\pi_0, \pi_1, \dots, \pi_{N-1}$ for the states of $RS = \{S_0, S_1, \dots, S_{N-1}\}$ by retrieving the rows of ∇Q^T (Qureshi et al., 1996; Stewart, 1994). Notice that the elements of any row of ∇Q^T are now present in the vertical segment files. Hence, to retrieve the elements of a certain row, we must retrieve its corresponding elements from all vertical segment files.

Remark 3.1. The history analysis of VS_l implies no computational operations applied on it, or no HA is applied on VS_l .

Remark 3.2. We only need to apply the GE method on a few number of columns instead of all columns (i.e. one vertical segment at a time) of the whole matrix. Therefore, we obtain a fewer number of new elements (zero elements are changed to be non-zero ones). This minimization has the advantage of substantially reducing the effect of the fill-in problem. Thus, our methodology is able to enhance the capability of RAM to accommodate large matrices.

From the previous discussion, we remark that the DBS technique manages both the RAM and HD during its run-time. This management may result in many problems affecting the overall performance negatively. We have implemented the DBS technique in an intelligent way to allow minimizing considerably the negative impact of these problems on the desired performance measures. In the following section, we discuss several of the implementation issues for the DBS technique.

4. Implementation of the DBS technique

In this section, we explain how the DBS technique is implemented to obtain the performance measures of the modeled system. To fulfill this objective, we have encountered several obstacles. The first of these obstacles stems from the enormous time consumed due to the back-and-forth movement of the vertical segment of non-zero elements during the RA and HA computations. In Section 4.1, we give a brief description of this obstacle. In addition, we present the technique that is utilized by the DBS technique to eliminate these movements. Further, since the RAM may suffer from overflow due to the exponential increase of the vertical segment of non-zero elements, we explain in Section 4.2 how the number of vertical segments K can be determined. Finally, in view of the fact that the history list will be accessed excessively during the HA of any vertical segment consuming a lot of time, Section 4.3 introduces an intelligent way for managing the records of the history list.

4.1. Implementation of the reduction and history analysis

The computation operations performed during the RA and HA of any vertical segment (e.g. VS_l) usually incorporate the following two processes. First, the insertion process of new elements among the elements of $VS_l \in VS$, e.g. Fig. 2c.2. Secondly, the deletion process of the elements situated under the path of the diagonal (i.e. change of non-zero elements into zeros), e.g. Fig. 2c.3. Insertion or deletion processes of elements in each $VS_l \in VS$ require back-and-forth displacements of these elements in the RAM. These displacement processes consume a lot of time. These processes represent the main drawback of the DBS technique. Therefore, in this

section, we illustrate how the RA and HA can be implemented in an intelligent way to allow eliminating these back-and-forth displacements.

To achieve this objective, we have been motivated by the *Successive Row Generation and Reduction (SRGR)* technique (Qureshi et al., 1996; Stewart, 1994). This technique transforms the whole matrix into its upper triangular form by reducing the rows, one by one, to their *Upper Triangular Row (UTR)* form. The zero elements of the UTR form can be defined for row_i as follows.

$$row_i(j) = 0 \quad \text{for } j < i \quad (4.1.1)$$

where $row_i(j)$ is the j th element of row_i . To transform any row_i into its UTR form, the SRGR technique retrieves row_i from the HD to the desired buffer. Then, the SRGR transforms the retrieved row into its UTR form by subtracting this row in a scaled form from the previous rows (row_l ; $1 \leq l \leq i-1$) stored in the RAM. The reduced form of row_i is then stored in the RAM.

In order to avoid handling whole rows of the matrix, we have used a modified version of the SRGR technique. We call it, the *MSRGR* technique. Let row portions denote the rows of each vertical segment. Let row_{il} ; $0 \leq i < N$ denotes the row portion of a vertical segment VS_l . The MSRGR technique helps us to reduce the row portions to its *Segmental Upper Triangular Row (SUTR)* form. The zero elements of row_{il} in its SUTR form are defined as follows.

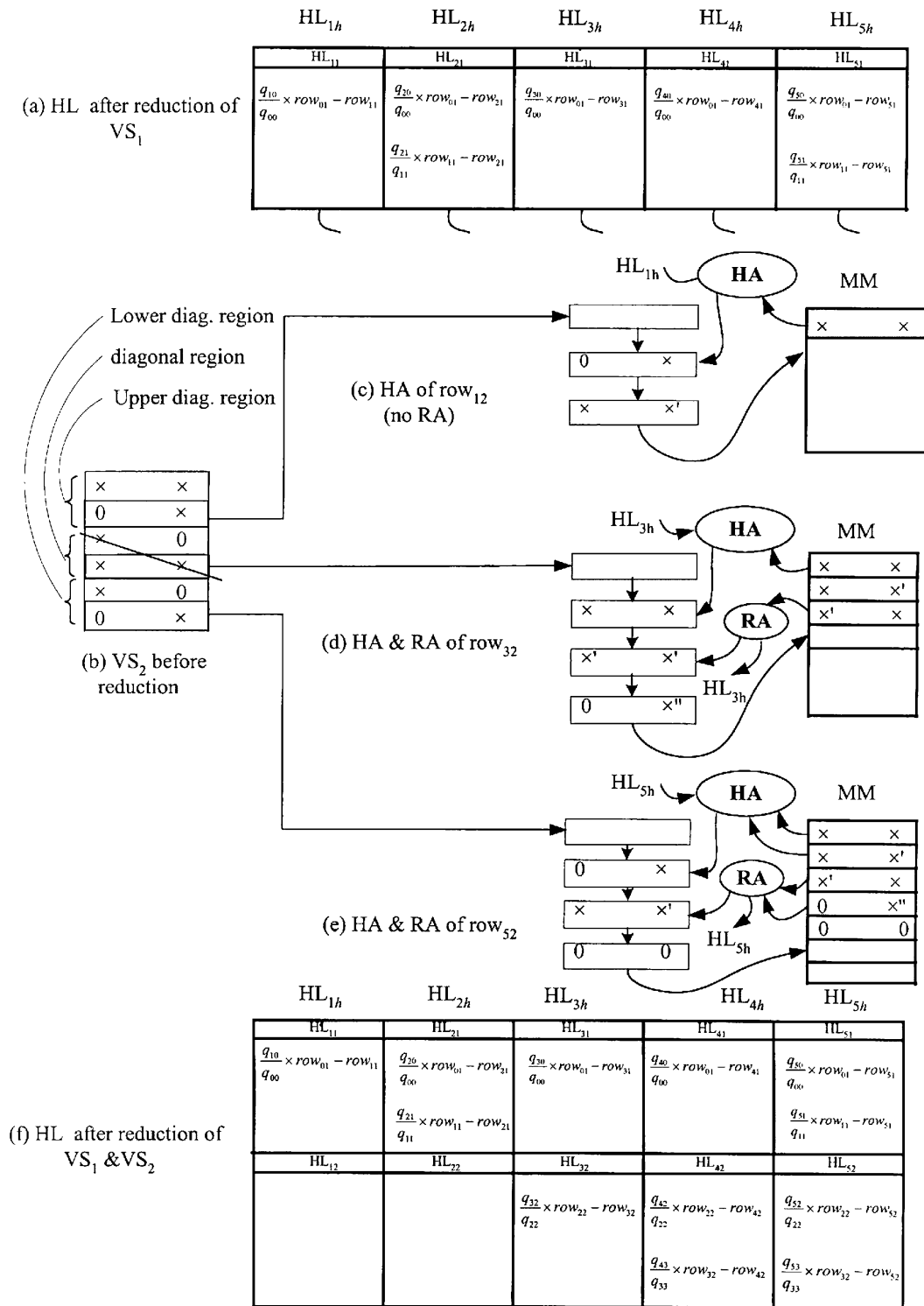
$$row_{il}(j) = 0 \quad \text{for } (j < i, Col_{bl} \leq j \leq Col_{el}) \quad (4.1.2)$$

where $row_{il}(j)$ is the element at the j th column of row_{il} . The definition of the SUTR form follows from the fact that each segment VS_l is constructed from a set of N row portions (i.e. $VS_l \equiv row_{il}$; $0 \leq i < N$) which extend from column Col_{bl} to column Col_{el} .

To achieve the transformation of rows of each vertical segment to its SUTR form, we first retrieve row_{il} from the HD to the desired buffer. Next, we perform on row_{il} the computation operations that have been done during the reduction processes of row_{i0} , row_{i1} , and $row_{i,(l-1)}$ to transform them to their SUTR forms. We call such operations, the *history analysis* of row_{il} . In this way, row_{il} is ready to be transformed into its SUTR form using the previously transformed rows (row_{0l} to $row_{i-1,l}$) that have been stored in the RAM. We call such transformation, the *reduction analysis* of row_{il} . After reducing the row_{il} into its SUTR form, we store it in the RAM.

Algorithm: Re-implementation processes of step 4 of the DBS technique (explained in Section 3) according to the MSRGR technique described above.

Step 4. Perform the following operations on the segments $VS_1, VS_2, VS_3, \dots, VS_K$ to transform the full matrix Q^T to its triangular form ∇Q^T . Note that Fig. 3

Fig. 3. Reduction of VS_2 using MSRGR technique.

shows the execution processes of the combined RA and HA for VS_2 of the matrix shown in Fig. 2a.

For $l = 1$ To K do

(a) Retrieve $row_{0l} \in row_0$ of VS_l from the HD to the RAM (see Fig. 3b for VS_2).

(b) Retrieve $row_{1l} \in row_1$ of VS_l from the HD to the reduction buffer (see Fig. 3b for VS_2).

(c) For $i = 2$ To $N - 1$, $l \geq 2$ do

(i) Apply the history analysis of row_{il} .

(ii) Transform the row_{il} to its SUTR form by applying reduction analysis.

- (iii) Transfer row_{il} from the reduction buffer to the RAM, as shown in Fig. 3.
- (iv) Register all computational operations performed during the RA of row_{il} in a *portion of the history list* (denoted by PHL_{il}), see Fig. 3b and f.

It is interesting to note that not all the regions of the row portions are subjected to RA. To clarify this statement, we define the following terms. Let the *upper diagonal region* be the region existing above the diagonal path through any vertical segment. Let the *diagonal region* be the region intersecting the diagonal path. Let the *lower diagonal region* be the region existing below the diagonal path. From our analysis, we have remarked the following interesting points. If we split the vertical segment VS_i into these three regions, the row portions of the upper diagonal region ($\text{row}_{il}; 0 \leq i \leq \text{Col}_{bl}$) will not be subjected to the reduction analysis. On the other hand, the row portions of the diagonal region ($\text{row}_{il}; \text{Col}_{bl} < i \leq \text{Col}_{el}$) and the lower diagonal region ($\text{row}_{il}; i > \text{Col}_{el}$) will be subjected to the reduction analysis. In Fig. 3c–e, we describe the application processes of the RA to the row portions: row_{12} , row_{32} , and row_{52} , as examples of these regions. The results of this application can be illustrated as follows. The result of the RA of the row portions of the lower diagonal region is the complete elimination of its elements as shown in Fig. 3e for row_{52} of VS_2 . On the other hand, the result of the RA of the row portions of the diagonal region is the partial elimination of its elements as shown in Fig. 3c for row_{32} of VS_2 .

4.2. Estimation of the suitable number of vertical segments

The proper operation of the DBS technique depends, largely, on the choice of the suitable K that prevents RAM overflow. This overflow occurs due to the enlargement of the vertical segment sizes during their transformation to their SUT forms. We have supported the DBS technique with a utility that allows the prediction of the final sizes of the vertical segments with the help of which a suitable K can be determined according to the available RAM. In this section, we describe the implementation of this utility. The problem of choosing K stems from the fact that the transformation of a vertical segment into its SUT form results in an exponential increase of its non-zero elements. These newly generated elements may cause RAM overflow, or the RAM may not be able to accommodate the enlarged size of some vertical segments. To avoid memory overflow, one may choose to increase K . When we increase the parameter K , the final sizes of the vertical segments after transformation decrease. This decrement in turn may prevent memory overflow. Unfortunately, increasing the parameter K leads to the increase of the time

needed to complete the solution due to the overhead processing needed to manage the history list file. Therefore the *minimum number* of vertical segments (denoted by $K_{\min}$) that prevents the memory overflow, must be determined carefully. Yet, choosing K requires the knowledge of the final sizes of vertical segments to determine whether the RAM will accommodate these sizes. Since the vertical segment's final forms are successive groups of columns of ∇Q^T , the estimation processes of the final sizes of vertical segments can be achieved by estimating the non-zero elements in all columns of ∇Q^T . We have made use of the following lemma to estimate the number of non-zero elements in any column j (denoted by Col_j).

Lemma. Let $e_{ij}; 0 \leq i, j \leq N - 1$ denotes the non-zero element of Col_j and row_i . Let u_j be the row index of the uppermost non-zero element of Col_j . Define the following formula as the set of non-zero elements of Col_j in the transition matrix Q^T .

$$\text{Col}_j \equiv \{e_{ij}; i \geq u_j\} \quad (4.2.1)$$

After applying the GE method to the matrix Q^T , the number of elements of any Col_j can be calculated by the following formula:

$$NE_j \cong |u_j - j + 1| \quad (4.2.2)$$

where u_j is the row index of the uppermost element of Col_j .

Proof. To prove this lemma, we use the simple SRN model shown in Fig. 4a. The matrix Q^T (14×14) generated from this model is shown in Fig. 4b. Simulate the effect of the GE method on the elements of the matrix Q^T shown in Fig. 4b. The GE method transforms the matrix Q^T ($N \times N$) into its upper triangular form in N steps. In the step i , the GE method uses row_i to eliminate the non-zero elements existing in Col_i under the diagonal element e_{ii} . Accordingly, to simulate the effect of the GE method on the elements of the matrix Q^T shown in Fig. 4b, we need to monitor the elimination operations that the GE method performs on the elements of the matrix Q^T in each of its N steps. Assume we first monitor the second step performed by the GE method on the elements of the matrix Q^T (i.e. eliminating the non-zero elements existing in the Col_1 under the diagonal element e_{11}). Actually, this step can be performed using row_1 to act upon the row_2 , row_3 , and row_6 in order to eliminate the set of elements $\{e_{21}, e_{31}, e_{61}\}$ existing under e_{11} . However, eliminating this set of elements induces the following elements: $\{e_{23}, e_{28}, e_{29}\}$ in row_2 , $\{e_{38}, e_{39}\}$ in row_3 , and $\{e_{68}, e_{69}\}$ in row_6 . A considerable part of these induced elements will be situated in the gap between the uppermost elements of Col_3 , Col_8 and Col_9 (i.e. e_{13} , e_{08} , e_{19}), and the corresponding diagonal elements (i.e. e_{33} , e_{88} , e_{99}). Fig. 4c shows the execution of

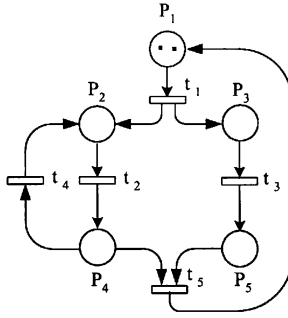


Figure 4-a: Simple SRN net.

$$Q^T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$
Figure 4-b: the Q^T matrix.

$$Q^T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 4-c: The effect of step 2 of GE.

Figure 4: Simulation of the effect of GE on Q^T

$$Q^T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 4-d: The effect of step 3 of GE.

$$VQ^T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$
Figure 4-e: The VQ^T matrix.Fig. 4. Simulation of the effect of GE on Q^T (a) simple SRN net, (b) the Q^T matrix, (c) the effect of step 2 of GE, (d) the effect of step 3 of GE, (e) the VQ^T matrix.

the second step performed by the GE method on the elements of the matrix Q^T , where e_{23} is generated in the gap between e_{13} and e_{33} . Also, $\{e_{28}, e_{38}\}$ will be induced in the position of zero elements present between e_{08} and e_{88} . Moreover, $\{e_{29}, e_{39}, e_{69}\}$ will be situated in the position of zero elements in the gap between e_{19} and e_{99} .

Finally, there are several non-zero elements that will be induced in the row₂ and row₃ during the elimination step $\{e_{21}, e_{31}\}$ due to the following reasons. The elements of the matrices $Q \equiv \{q_{ij}\}_{N \times N}$ (or Q^T) resulting from systems modeled by the Petri net (PN) theory exist near the diagonal (Chiola et al., 1995; Hirel et al., 2000). Thus, most elements of any row of the matrix Q (or Q^T) will exist near the matrix diagonal. Consequently, the non-zero elements of any column will exist in several neighboring rows. Subsequently, applying any step i of the GE method on the matrix Q^T will induce non-zero elements in several neighboring rows to row _{i} . A part of these induced elements will reduce the gap between the uppermost elements and the corresponding diagonal elements, while the other part will be situated under the matrix diagonal to be eliminated through subsequent elimination steps. Hence, we deduce that the third step performed by the GE method on the matrix Q^T will induce new non-zero elements in neighboring rows to row₃. Moreover, the elements induced in row₃, during the second step of the GE method, will produce more non-zero elements in the gap between the uppermost

elements of row₃, row₈, and row₉ as well as the corresponding diagonal elements. Fig. 4d shows the execution of step 3 of the GE method on the matrix Q^T , where e_{28} (induced in step 2 of the GE method) produces e_{48} and e_{78} . Also, e_{29} (induced in step 2 of the GE method) produces e_{49} and e_{79} . This process further tightens the gap between e_{08} and e_{88} as well as e_{19} and e_{99} .

Since the execution of GE steps gradually reduces the gaps between the uppermost elements and the corresponding diagonal elements, the number of non-zero elements in Col _{i} of VQ^T can be calculated by the following formula.

$$NE_i \cong |u_i - i + 1|$$

where u_i is the row index of the uppermost element of Col _{i} . $\square$

From our experience with the SRN models and their generated Q^T matrices, we have found that the estimating processes of the 14×14 matrix are still valid for the Q^T matrices generated from most SRN system models. This result is due to the previously mentioned property: the elements of the Q^T matrices resulting from the PN models exist near the diagonal. In Section 5, we illustrate the validity of our estimating processes of large Q^T matrices.

From our analysis, we have found that the calculation of NE_i for any Col _{i} of VQ^T depends only on its

uppermost elements. Thus, we can easily get a set $\mathbf{NE} = \{NE_0, NE_1, \dots, NE_{N-1}\}$, where NE_i carries the number of non-zero elements in the corresponding column of $\nabla \mathbf{Q}^T$. Having the set $\mathbf{NE}$, one can calculate the minimum number of vertical segments ($K_{\min}$) that can be accommodated by a given RAM. In this algorithm, we take into consideration that any non-zero element of a vertical segment needs 12 bytes of memory (4 bytes for the column index, and 8 bytes for the element storage). The following algorithm illustrates how the set $\mathbf{NE}$ supports the calculation of the parameter $K_{\min}$.

Algorithm: Calculation of $K_{\min}$ for a given RAM.

Step 1. Initialize the minimum number of vertical segments such that $K_{\min} = 1$.

Step 2. Calculate the number of non-zero elements in vertical segments ($no_elem_VS_i$) through the following loop.

For $i = 1$ To $K_{\min}$

(i) Calculate Col_{bi} and Col_{ei} of VS_i according to $no_col_VS = \lfloor N/K_{\min} \rfloor$

where no_col_VS is the number of columns of a vertical segment VS_i . $\lfloor x \rfloor$ is the maximum integer less than x .

Hence

$$Col_{bi} = (i - 1) \times no_col_VS$$

$$Col_{ei} = i \times no_col_VS - 1, \quad \text{for } i < K_{\min}$$

$$Col_{ei} = N - 1, \quad \text{for } i = K_{\min}$$

(ii) Calculate $no_elem_VS_i$ according to the following equation:

$$no_elem_VS_i = \sum_{l=Col_{bi}}^{Col_{ei}} NE_l$$

If $(no_elem_VS_i \times 12) \geq \text{RAM size}$,

Then Go To Step 3

Else $K_{\min} = K_{\min} + 1$

• $i = 1$

• Go To Step 2

Step 3. Display $K_{\min}$.

4.3. Management of history analysis

As we have explained in Section 4.1, the HA must be performed on each row portion. Consequently, during the HA of row portion row_{il} , we must perform on row_{il} all the computational operations done during the RA of the previous row portions $\mathbf{S} \equiv \{row_{ij}; 1 \leq j \leq (l - 1)\}$. On the other hand, many of these computational operations may not be needed during the HA of a row portion. Since the records of these computational operations are present in the form of the history list (Section 3) on the HD, the determination

processes of their applicability need to retrieve them from the HD.

In this section, we present the *management technique* utilized by the HA to guess the applicable operations without the need to access the HD. The main advantage of the proposed management technique is the elimination of time consumed during the accessing processes of the HD for inapplicable operations. In addition, we get rid of the burden of performing these inapplicable operations. To clarify how our proposed management technique performs the HA of row portions of a vertical segment, let row_{im} be one of the row portions of VS_m . Also, we assume that the HA for the row_{im} is required. It is interesting to note that the row_{im} may be situated in the upper diagonal region, or the diagonal region, or the under diagonal region with respect to VS_m . The position of row_{im} greatly affects the way of handling its HA by the proposed management technique. In the following, we illustrate how the proposed management technique handles the history list records corresponding to the row portions of the previously mentioned regions. For this purpose, let us define the following. Let the *non-zero row portion* be the row portion that some of its elements are non-zero. Let the *zero row portion* be the row portion that some of its elements are zero.

To illustrate how the HA of the zero row portion and the non-zero row portion can be managed, suppose that the row_{im} is situated in the upper diagonal region of VS_m (i.e. $0 \leq i < Col_{bm}$). The HA of the non-zero row_{im} can be illustrated throughout applying on row_{im} all the computational operations registered in the history list corresponding to row_{ij} ; $0 \leq j \leq m - 1$. From our experience with the matrix $\mathbf{Q}$ generated from the PN system models, we have found that the non-zero elements of any row in this $\mathbf{Q}$ are confined to the area near the matrix diagonal. Also, each row_i has a set of leftmost zero row portions given by the set $\mathbf{A} \equiv \{row_{ij}; 0 \leq j < l_i\}$, where row_{il_i} is the leftmost non-zero row portion of row_i .

Since the row portions of $\mathbf{A}$ are zero row portions, the members of $\mathbf{A}$ will not be subjected to any RA. In turn, no computational operations will be registered in the history list corresponding to the set $\mathbf{A}$. On the contrary, we expect that the history list will include records of computational operations corresponding to the non-zero row portions of the set $\mathbf{B} \equiv \{row_{ij}; l_i \leq j < d_i\}$, where row_{id_i} is the non-zero row portion situated in the diagonal region of VS_{d_i} (i.e. $Col_{bd_i} \leq i \leq Col_{ed_i}$). This expectation is due to the fact that all the members of $\mathbf{B}$ are situated under the matrix diagonal. Hence, $\mathbf{B}$ will be subjected to RA.

From the above discussion, we conclude the following. During the HA of row_{im} , we only need to apply on row_{im} the set of operations stored in $\mathbf{S} \equiv \{PHL_{ij}; l_i \leq j < d_i\}$, where PHL_{ij} is a part of the history list carrying the

group of computational operations performed on row_{ij} during its RA (as shown in Section 4.2). Having the set S , we can define the *history domain* of row_{im} (denoted by Dom_{im}) as follows.

$$\text{Dom}_{im} \equiv \{j; l_i \leq j \leq d_i\} \quad (4.3.1)$$

We can then designate l_i and d_i as the left and right history domain sides of row_{im} and denote them as $L\text{Dom}_{im}$ and $R\text{Dom}_{im}$ respectively. The HA of $\text{row}_{im} \in \{\text{row}_{jm}, 0 \leq j < u_m\}$ implies all the computational operations that have been performed on row_{im} . These operations have been stored in the history list corresponding to row_{im} . As shown in Section 4.1, these computational operations have been done between row_{im} and the rows higher than row_{im} (i.e. between row_{im} and row_{jm} ; $1 \leq j < i$). Since row_{im} and the rows higher than row_{im} are all zero row portions, the history domain of row_{im} ; $0 \leq i < u_m$ can be assumed to be equal to ϕ , where ϕ is the empty set. Thus, the history domain of the zero row portions extending higher than the uppermost row portion of VS_m is equal to ϕ .

In the following, we illustrate the processes of the HA of zero row_{im} . To perform these processes, we use the same processes that have been applied on the non-zero row_{im} . In this case, l_i and d_i will be also the history domain sides of row_{im} . Throughout the HA of zero row_{im} , we need to apply on row_{im} the computational operations stored in S . Yet, the state of row_{im} will be only changed if the computational operations stored in PHL_{ij} ; $l_i \leq j \leq d_i$ are performed using the non-zero row portions. This constraint can be accomplished by redefining the history domain of the zero row_{im} as follows.

$$\text{Dom}_{im} \equiv \{j; l_i \leq j \leq d_i, \text{Col}_{ej} \geq u_m\} \quad (4.3.2)$$

The PHL_{ij} ; $l_i \leq j < d_i$ contains the records of computational operations performed between the members of row_{km} ; $\text{Col}_{bj} \leq k \leq \text{Col}_{ej}$ and the row_{im} . Consequently, the condition $\text{Col}_{ej} \geq u_m$ guarantees that the set row_{km} ; $\text{Col}_{bj} \leq k \leq \text{Col}_{ej}$ contains the non-zero row portions. This condition acquires a great validity due to the following reason. From our experience with the matrix Q generated from the PN system models, most of the non-zero elements of a vertical segment are concentrated in the diagonal region rows. The presence of these elements decreases significantly as we move up and down through the rows of the upper and lower diagonal regions, respectively. Moreover, in large matrices of practical systems, the non-zero elements vanish completely in most rows of the upper and lower diagonal regions.

Having finished from the analysis of the upper diagonal region, suppose the row_{im} is situated in the diagonal region of VS_m (i.e. $\text{Col}_{bm} \leq i \leq \text{Col}_{em}$). According to Eqs. (4.3.1) and (4.3.2), we can easily derive the history domain of row_{im} as follows.

$$\text{Dom}_{im} \equiv \{j; l_i \leq j \leq m-1\} \quad (4.3.3)$$

It must be noted that all the row portions of the diagonal region are of the non-zero kind. Hence, there is no need to discuss the zero row portions.

Finally, suppose row_{im} is situated in the lower diagonal region of VS_m (i.e. $\text{Col}_{em} < i < N$). Row portions of these regions can be non-zero or zero type. The history domain of non-zero row_{im} is the same as that of Eq. (4.3.3). The history domain of zero row_{im} is defined as follows.

$$\text{Dom}_{im} \equiv \{j; l_i \leq j \leq m-1, \text{Col}_{ej} \geq u_m\} \quad (4.3.4)$$

In the following algorithm, we explain how our management technique uses Eqs. (4.3.1)–(4.3.4) to perform the HA of row_{im} .

Algorithm: History analysis of row_{im} of VS_m (Dom_{im})

Step 1. Determine the Dom_{im} of row_{im} through the following steps.

- (a) If $(0 \leq i < u_m)$ (the upper diagonal region), then

$$\text{Dom}_{im} = \phi$$
- (b) If $(u_m \leq i < \text{Col}_{bm})$ (the upper diagonal region), then
 - (i) Dom_{im} of zero $\text{row}_{im} \equiv \{j; l_i \leq j \leq d_i, \text{Col}_{ej} \geq u_m\}$
 - (ii) Dom_{im} of non-zero $\text{row}_{im} \equiv \{j; l_i \leq j \leq d_i\}$.
- (c) If $(\text{Col}_{bm} \leq i)$ (the diagonal region and the lower diagonal region), then
 - (i) Dom_{im} of zero $\text{row}_{im} \equiv \{j; l_i \leq j \leq m-1, \text{Col}_{ej} \geq u_m\}$
 - (ii) Dom_{im} of non-zero $\text{row}_{im} \equiv \{j; l_i \leq j \leq m-1\}$.

Step 2. Use the Dom_{im} of row_{im} to apply on row_{im} the set of operations $A \equiv \{PHL_{ij}; j \in \text{Dom}_{im}\}$ where PHL_{ij} is a part of the history list carrying the computational operations performed during the RA of row_{ij} .

5. Numerical results

To illustrate the merits of using the DBS technique, we have applied the DBS on the SRN model of the Kanban manufacturing system shown in Fig. 5. This model has been used previously in the literature to show the effectiveness of several largeness tolerance techniques (Deavours and Sanders, 1998a; Deavours and Sanders, 1998b). The SRN Kanban model is composed of four subnets (Donatelli and Kemper, 2001). At each subnet, a token enters, spends some time, and exits or restarts with certain probabilities. Once the token leaves the first net, it may enter the second or third subnet. The token must go through the fourth subnet to leave the system.

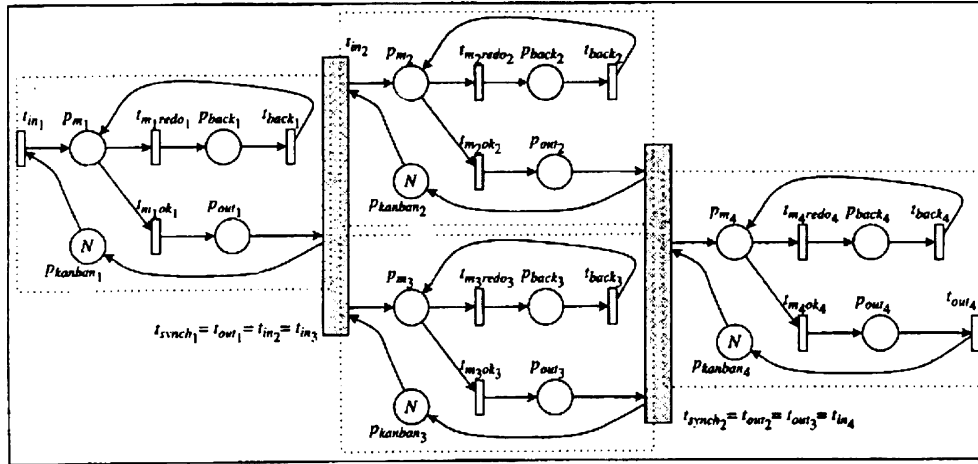


Fig. 5. The model of the Kanban manufacturing system.

To solve the SRN Kanban model of Fig. 5, we have represented the synchronization transitions by the timed transitions. Assuming that the places p_{kanban_1} , p_{kanban_2} , p_{kanban_3} and p_{kanban_4} contain 3, 2, 2 and 2 tokens respectively, we get the matrix Q^T of dimension $N \times N$, where $N = 18,400$ as shown in Fig. 6. In this figure, the presence of the matrix elements near the diagonal can be observed. Moreover, suppose that the matrix Q^T cannot be solved using the traditional iterative methods due to the stiffness of its rates. Consequently, the matrix Q^T can be managed only by using the direct methods such as the GE technique. Unfortunately, applying the GE technique on the matrix Q^T causes memory overflow even for 256 MB of RAM (which is a common resource for many platforms). If the algorithm of Section 4.2 supports the DBS technique, the matrix Q^T can be solved using the DBS technique for $K \geq 5$ due to the following. The estimated memory used by the DBS technique is more than 256 MB, which is beyond the available memory. Consequently, we have applied the DBS technique on the matrix Q^T for the values of K ranging from 20 to 200. For these variations of K , we

have measured the time needed by the DBS technique to solve the matrix Q^T .

All the experiments done using the DBS technique are performed on an Intel Pentium III-550 MHz with 256 MB of RAM and 20 GB of HD. Moreover, for a certain number of vertical segments K , the size of the vertical segment SUT form varies from one vertical segment to another as shown in Fig. 7 for $K = 40$. Therefore, we are interested in monitoring the maximum size of these SUT forms. By allocating enough memory for this maximum size (denoted by $M_{max,K}$), the DBS technique has the capability to completely convert all the vertical segments to their SUT forms. Table 1 shows the time (denoted by T_K) and the $M_{max,K}$ needed by the DBS technique to completely convert the vertical segments into their corresponding SUT forms for $20 \leq K \leq 200$.

It is interesting to note that the proposed DBS technique has been developed based on the modification of execution sequences of the GE procedure. Therefore for a given K , the size of the vertical segment SUT form (denoted by M_i ; $1 \leq i \leq K$) is related to the size of the matrix Q^T (denoted by $M_{\nabla Q}$) by the following relation.

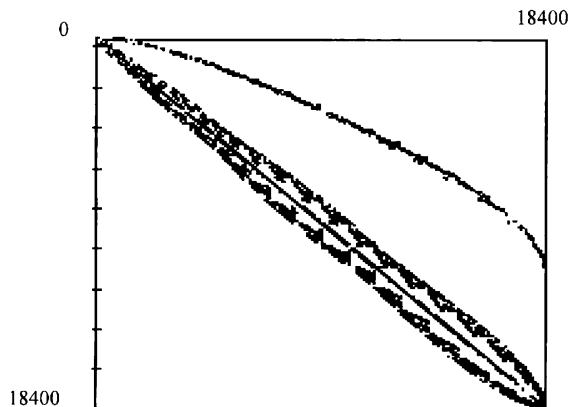
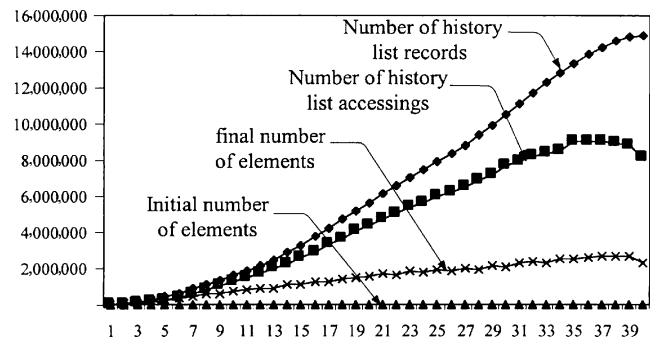
Fig. 6. The $18,400 \times 18,400$ Q^T of the Kanban model.Fig. 7. Number of history list records and history list accessings, initial and final number of elements for vertical segments resulting from applying DBS for $K = 40$.

Table 1

Time (T_K), maximum memory (M_K), percentage increase in time ($\%T_K$) and percentage reduction in memory ($\%M_K$) consumed by the DBS for $10 \leq K \leq 200$

K	0	10	20	40	60	80	100	120	140	160	180	200
T_K (s)	13683	14863	15221	16302	17676	20209	20916	21135	22957	23266	24711	27480
$\%T_K$	0	8.62	11.24	19.14	29.18	36.92	52.86	54.46	67.78	70.04	80.60	100.83
$M_{\max,K}$ (MB)	675.5	119.42	61.04	30.61	20.64	15.57	12.50	10.50	8.99	7.82	8.99	6.26
$\%M_K$	0	82.32	90.96	95.47	96.94	97.70	98.15	98.44	98.67	98.84	98.67	99.07

$$M_{\nabla Q} = \sum_{i=1}^K M_i \quad (5.1)$$

Using Eq. (5.1), the $M_{\nabla Q}$ has been found to be 675.5 MB (i.e. 59, 025, 755 non-zero elements). The enormous size of this matrix $\mathbf{Q}^T$ indicates why the GE technique is not able to solve such a matrix. To illustrate the advantage of using the proposed DBS technique, we have calculated the reduction percentage in RAM needed by the DBS (denoted by $\%M_K$) for all variations of K . The percentage $\%M_K$ can be obtained using the following formula.

$$\%M_K \equiv \frac{(M_{\nabla Q} - M_{\max,K})}{M_{\nabla Q}} \times 100 \quad (5.2)$$

In Table 1, we illustrate the values of $\%M_K$ when $20 \leq K \leq 200$. From Table 1, we have remarked the following interesting points.

- (i) $\%M_K$ increases as K increases;
- (ii) For $K > 60$, the change in $\%M_K$ is less than 1%; and
- (iii) When $K = 60$, $\%M_K$ reaches 96%.

Hence it will be satisfactory to use the DBS technique for $K \leq 60$. Regrettably, the increase in the value of $\%M_K$ is accompanied by an increase in the corresponding solution time (denoted by T_K) consumed by the DBS technique to solve the matrix $\mathbf{Q}^T$. In the following, we calculate the percentage of increasing T_K (denoted by $\%T_K$).

$$\%T_K \equiv \frac{(T_K - T_0)}{T_0} \times 100 \quad (5.3)$$

where T_K ($20 \leq K \leq 200$): the time consumed by the DBS technique to solve the matrix $\mathbf{Q}^T$; T_0 ($K = 0$): the time consumed by the GE technique to solve the matrix $\mathbf{Q}^T$. The behavior of the DBS technique is the same as that of the GE technique when $K = 0$.

The advantages gained from our DBS technique can diminish greatly if $\%T_{60}$ has a comparable value to 96% of $\%M_{60}$. Yet, we have not been able to get T_0 because of the enormous size of the matrix $\mathbf{Q}^T$ preventing its solution, as previously stated. As an approximate solution for this problem, we have calculated T_0 by *extrapolating* the set $\{T_K; 20 \leq K \leq 200\}$, for $K = 0$. To make our extrapolation more accurate, we have also measured T_K for $K = 10$. Having T_0 , we can compute $\%T_K$ for

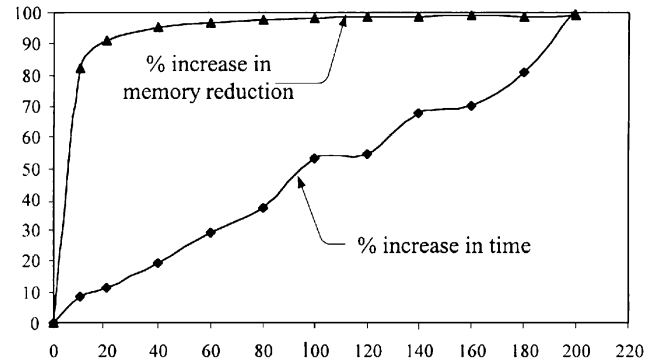


Fig. 8. % Increase in memory reduction, % increase in time for K of the DBS varying from 20 to 200.

$20 \leq K \leq 200$ (see also Table 1). We have plotted in Fig. 8, the change of $\%T_K$ and $\%M_K$ for $20 \leq K \leq 200$. From Fig. 8, we can remark the following points. For $K = 60$, while $\%M_K$ reaches 96%, $\%T_K$ reaches only 20%. When $K > 60$, $\%T_K$ increases excessively and $\%M_K$ increases slightly (i.e. less than 1%). When $K \leq 60$, there is a remarkable difference between $\%T_K$ and $\%M_K$ due to the effect of the proposed history management technique (discussed in Section 4.3).

To understand the effect of the proposed history management technique on the performance of the DBS, we must notice that the main difference between the GE and DBS techniques is the time consumed during the HD accessing. A part of the HD accessing occurs during the HA (denoted by ACC_{HA}). Another part occurs during the retrieval and storage of the vertical segment elements (denoted by ACC_{RS}). From our analysis, the value of ACC_{RS} is nearly constant during the variation of K due to the following reasons. The total number of the vertical segment elements is constant. Also, this number does not vary with K (see Eq. (5.1)). The operations (stored in the history list) performed during the HA are the main cause of these elements. Thus, the total number of history list records accessed will be more or less constant. Subsequently, minimizing ACC_{HA} will greatly enhance the performance of the DBS technique. Indeed, we can perform the HA by accessing all the history list records. Yet, we need to access the HD several times.

Fig. 7 shows the number of times the history list is accessed by the proposed history management technique

for $K = 40$. From this figure, we can remark the difference between the number of history list records and that of the history list records accessed for the vertical segments of $K = 40$. To access the entire history list records, we need to access the HD 258, 294, 947 times (i.e. $ACC_{HA} = 258, 294, 947$) as shown in Fig. 7. On the other hand, the proposed history management technique (explained in Section 4.3) needs to access the HD 180, 532, 385 times (i.e. $ACC_{HA} = 180, 532, 385$). Thus, the proposed history management technique achieves 30% reduction in ACC_{HA} . Moreover, ACC_{HA} is usually much greater than ACC_{RS} . For example, from Fig. 7, we found that while ACC_{HA} is equal to 180, 532, 385, ACC_{RS} equals 59,150,755. We can then state that ACC_{HA} constitutes nearly 75% of the time consumed during accessing the HD. Hence, we can conclude that ACC_{HA} affects greatly the performance of the DBS technique (i.e. time of solution). For each vertical segment $\{VS_i; 1 \leq i \leq K\}$, ACC_{HA} is controlled by the history domain range of the vertical segment VS_i (denoted by $DomR_i$).

$$DomR_i = \max\{LDom_{ij}; 0 \leq j \leq N - 1\} - \min\{RDom_{ij}; 0 \leq j \leq N - 1\} \quad (5.4)$$

Fig. 9 illustrates that there is a proportional effect for the $DomR_i$ on the solution time of the vertical segments $\{VS_i; 1 \leq i \leq K\}$ when $K = 40$. The effect of $DomR_i$ on the solution time of VS_i ensures the previous conclusion concerning the effect of ACC_{HA} on the performance of the DBS technique due to the following reason. The increase of $DomR_i$ means the increase of the accessed history list records for any vertical segment VS_i .

To gain more insight about the operation of the proposed management technique, we use the following example. Let us monitor the HA of VS_{10} and that of VS_{30} during the application of the DBS technique when $K = 40$. As shown in Fig. 9, $DomR_{10} = 7$ and $DomR_{30} = 17$. The number of times the HD is accessed during the execution process of the HA of VS_{10} and that of VS_{30} on the set $PHL_i; 1 \leq i \leq 40$ when $K = 40$ is shown

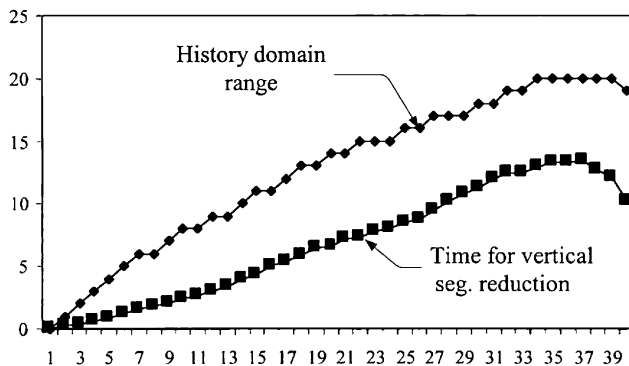


Fig. 9. Time for reduction and history domain range of vertical segments for $K = 40$.

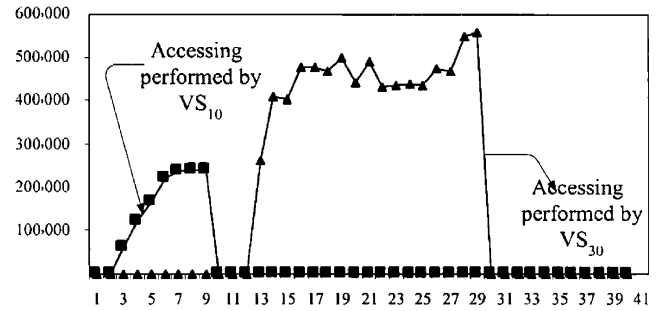


Fig. 10. Number of accessings performed by VS_{10} and VS_{30} on $\{HL_i; 1 \leq i \leq 40\}$ for $K = 40$.

in Fig. 10. From our analysis of Figs. 9 and 10, we have observed the following. These accessing processes are performed on history list portions situated within the domain range of VS_{10} and that of VS_{30} (i.e. $\{PHL_i; 10 - DomR_{10} \leq i < 10\}$, $\{PHL_i; 30 - DomR_{30} \leq i < 30\}$). The number of times the history list records is accessed corresponding to nearby vertical segments is greater than that of farther vertical segments. For example, during the HA of VS_{30} , the accessing process to PHL_{29} is greater than that to PHL_{28} or PHL_{27} . This behavior is due to the fact that the non-zero elements of a certain row extend through several neighboring vertical segments. Subsequently, during the HA of VS_i , the history list of neighboring vertical segments will contain more applicable operations to row portions of VS_i than that of farther VS_i .

Furthermore, it is interesting to illustrate how the proposed management technique performs the HA of a row portion, as a basic operation of this technique. Fig. 11 shows the change of the left and right history domain sides for the row portions of VS_{10} and those of VS_{30} for $K = 40$, (i.e. $LDom_{ij}, RDom_{ij}; 0 \leq i \leq N - 1, j \in (10, 30)$). The effect of the matrix diagonal on these history domain sides can be noticed by the descending zigzag appearance of the diagrams shown in Fig. 11. Moreover, as shown in Fig. 12, the history domain of the row

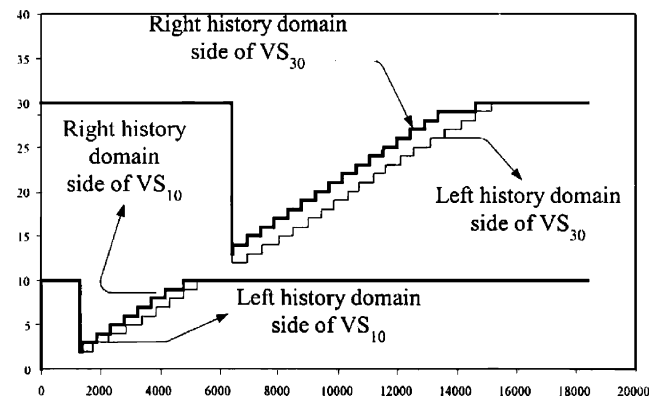


Fig. 11. Left and right history domain sides for row portions of VS_{10} and VS_{30} for $K = 40$.

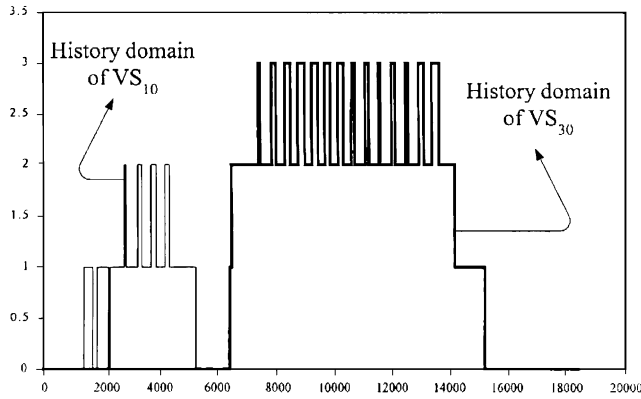


Fig. 12. History domain for row portions of VS_{10} and VS_{30} for $K = 40$.

portions VS_{10} and that of VS_{30} has not exceeded 2 for VS_{10} and 3 for VS_{30} . Also, we can notice the close relation between Figs. 11 and 12 according to Eqs. (4.3.1)–(4.3.4). This close relation is due to the fact that the elements of the matrix Q^T are confined to the area near the matrix diagonal. Subsequently, the leftmost row portion row_{i,l_i} of row_{ij} is situated nearby the vertical segment VS_{l_i} (i.e. $(j - l_i) \ll K$). Furthermore, the number of times the HD is accessed by each row portion on the history list is presented in Fig. 13. The effect of $Dom_{10,i}$ and $Dom_{30,i}$ on the number of times the history

list is accessed can be noticed by comparing Figs. 12 and 13. From this comparison, the row portions that have an equal history domain access the history list in nearly equal number of times. Also, the row portions that have a greater history domain access the history list more times.

Using the proposed history management technique, the HA greatly affects the number of non-zero elements enclosed in a vertical segment VS_i . From Fig. 14, we can observe the exponential increase in the number of elements enclosed in VS_{10} and that of VS_{30} . Fig. 14 illustrates the effect of the application of the DBS technique on the final number of elements that have been enclosed in the row portions of VS_{10} and that of VS_{30} . As shown in Fig. 14, the number of non-zero elements has reached an average value of 200 and 280 for row portions of VS_{10} and VS_{30} , respectively. On the contrary, the initial number of elements enclosed in the row portions of VS_{10} and that of VS_{30} (before application of the DBS technique) has not exceeded 6 for the row portions of VS_{10} and those of VS_{30} . The curve shown in Fig. 14 increases exponentially due to the application of the HA on the three previously mentioned regions of VS_{10} and those of VS_{30} . Moreover, we can observe from Fig. 14 that the non-zero elements are only distributed throughout the row portions of the upper diagonal and the diagonal

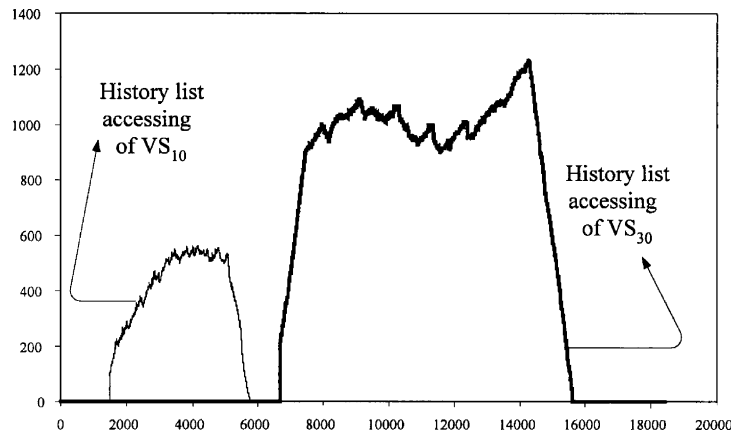


Fig. 13. History list accessings performed during HA of row portions of VS_{10} and VS_{30} for $K = 40$.

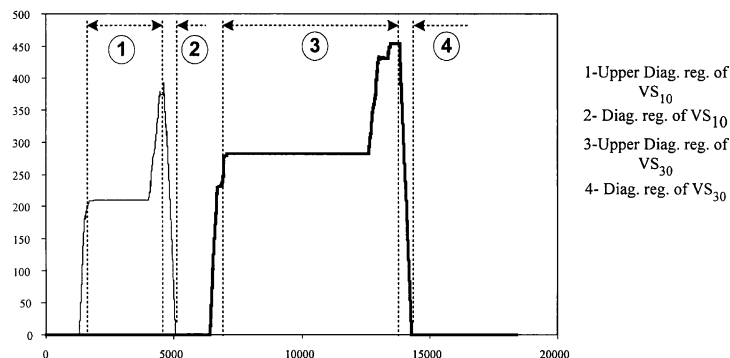


Fig. 14. Final number of elements in row portions of VS_{10} and VS_{30} for $K = 40$.

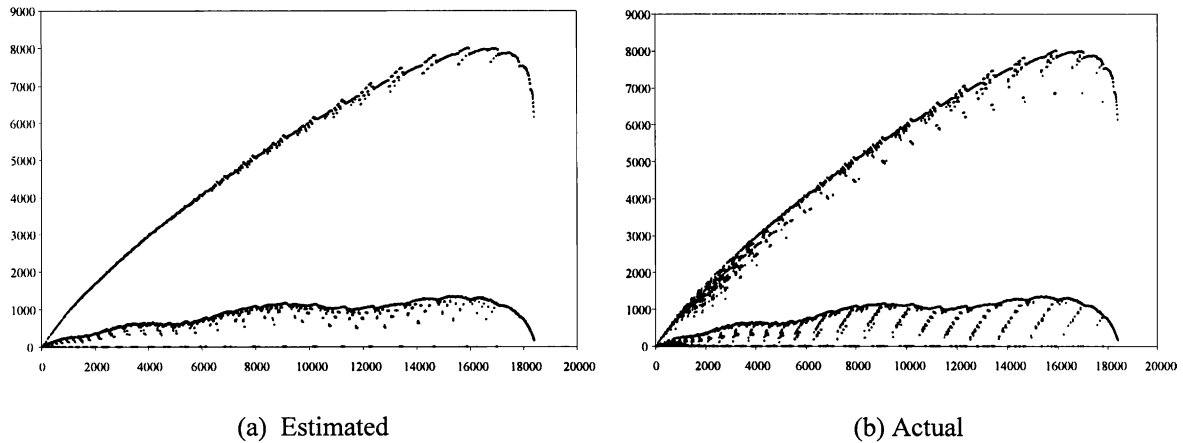


Fig. 15. Estimated and actual number of elements in columns of ∇Q^T .

regions, while the non-zero elements present in the row portions of the under diagonal region are eliminated through the RA. Also, these non-zero elements are only present in a small subset of the row portions due to the property of the matrices that can be obtained from the PN models as previously mentioned.

From our analysis of Fig. 14, we can also observe that the final number of elements has a peak value for the set of row portions existing in the border between the upper diagonal and the diagonal regions. This peak in the number of elements is attributed to the peak present in the number of times the history list is accessed as shown in Fig. 13. The presence of this peak is due the distribution of the elements around the diagonal in the border between the upper diagonal and the diagonal regions. This distribution means the existence of many applicable records in the history list for row portions existing in the border between the upper diagonal and the diagonal regions. Consequently, applying the HA of row portions of the border will induce many elements for these row portions. The effect of RA can be shown in the sharp inclination in the number of elements for row portions of the diagonal region.

Finally, the effectiveness of the estimation criteria proposed in Section 4.2 can be observed by comparing Fig. 15a and b. In Fig. 15a, the number of elements in each column of ∇Q^T is estimated according to the lemma of Section 4.2. In Fig. 15b, the actual number of elements in each column of ∇Q^T is monitored from the columns of the vertical segments in their SUT forms. The resemblance between the two figures can be easily observed, where the error in estimation has not exceeded 0.001% for the 59,025,755 elements of ∇Q^T .

6. Conclusions

In this paper, we have presented the DBS technique as a largeness tolerance technique based on a modified

version of the GE procedure. The basic concept of the DBS technique depends on splitting the whole matrix into a number of vertical segments. Each vertical segment consists of a band of successive columns. Each vertical segment is acted upon by a modified version of the GE procedure in such a way that these segments are transformed and become a part of the triangular form of the original matrix. Further, we store these segments in the hard disk to be used in calculating the steady state probabilities. Moreover, a list of the computational operations (performed during the transformation of these vertical segments) is stored on the hard disk. By varying the number of vertical segments, the memory needed by the DBS technique is greatly minimized.

To minimize the DBS solution time, we have supported the DBS by two novel techniques. The first technique enables the DBS to get rid of the time consumed due to the back and forth movements of the non-zero elements of vertical segments during the transformation processes of each of these vertical segments into its SUT form. The second technique permits clever management of the disk accessing for the purpose of computation operation records. By making use of these two techniques, we have the capability of minimizing effectively the DBS solution time. Moreover, since the number of elements of any vertical segment increases excessively during its transformation to the corresponding SUT form, we have proposed a utility for the purpose of allowing an approximate estimation for the number of vertical segments to prevent the memory overflow.

To illustrate the merits of using the DBS technique, we have applied the DBS on the matrix Q obtained from the SRN model of the Kanban manufacturing system. From this model, we have produced 18,400 states to describe the behavior of large-scale models. Also, we have presented the detailed results concerning time and space requirements of the DBS technique. These results show that varying the number of vertical segments leads

to 96% reduction in the memory required as compared to that needed by the GE technique, with just only 20% increases in the solution time. This reduction is due to the fact that the GE technique needs 675 MB to solve the state-transition-rate of this model but the DBS technique needs only 60 MB, with 20% increases in the solution time. Finally, we have analyzed the execution processes of the disk management technique to clarify its effectiveness.

References

- Abdallah, H., Hamza, M., Arabnia, H.R., 2000. Sensitivity computation of the expected accumulated reward of stiff Markov models. in: *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications*, Monte Carlo Resort, Las Vegas, Nevada, USA, vol. 3, pp. 1415–1421.
- Buchholz, P., Ciardo, G., Kemper, P., Donatelli, S., 2000. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS Journal on Computing* 13 (3), 203–222.
- Buchholz, P., Kemper, P., 2001. Quantifying the dynamic behavior of process algebras. In: *Proceedings of the Joint PAMP-PROBMIV Workshop*, Aachen, Germany Lecture Notes in Computer Science, vol. 2165. Springer-Verlag, pp. 184–199.
- Buchholz, P., 2001. Hybrid analysis of SGSPNs with time-dependent transition rates. *Performance Evaluation* 44 (1–4), 187–209.
- Buchholz, P., Kemper, P., 2001. Compact representations of probability distributions in analysis of superposed GSPNs. In: *Proceedings of the 9th International Workshop on Petri Nets and Performance Models*, Aachen, Germany. IEEE CS Press, pp. 81–90.
- Chiola, G., Franceschinis, G., Gaeta, R., Ribaud, M., 1995. GreatSPN 1.7: Graphical editor and analyzer for timed and stochastic petri nets. *Performance Evaluation* 24 (1–2), 47–68.
- Deavours, D.D., Sanders, W.H., 1998a. An efficient disk-based tool for solving large models. *Performance Evaluation* 33 (1–2), 67–84.
- Deavours, D.D., Sanders, W.H., 1998b. On-the-fly solution techniques for stochastic Petri nets and extensions. *IEEE Transactions on Software Engineering* 24 (10), 889–902.
- Donatelli, S., Kemper, P., 2001. Integrating synchronization with priority into a Kronecker representation. *Performance Evaluation* 44 (1–4), 73–96.
- German, R., 2001. Iterative analysis of Markov regenerative models. *Performance Evaluation* 44 (1–4), 51–72.
- Goseva-Popstojanova, K., Trivedi, K.S., 2000. Stochastic modeling formalisms for dependability, performance and performability. In: *Lecture Notes in Computer Science*, vol. 1769. Springer-Verlag, pp. 403–422.
- Gribaudo, M., Sereno, M., Horvath, A., Bobbio, A., 2001. Fluid stochastic Petri nets augmented with flush-out arcs: modelling and analysis. *Discrete Event Dynamic Systems: Theory & Applications* 11 (1–2), 97–117.
- Haverkort, B.R., Boudewijn, R., Alexander, O., 1997. Steady-state Analysis of Infinite stochastic Petri nets: comparing the spectral expansion and the matrix-geometric method. In: *Proceedings of the 7th International Workshop on Petri Nets and Performance Models*, Saint Malo, France, pp. 36–45.
- Hermanns, H., Herzog, U., Katoen, J.-P., 2002. Process algebra for performance evaluation. *Theoretical Computer Science* 274 (1–2), 43–87.
- Hirel, C., Trivedi, K.S., Tuffin, B., 2000. SPNP Version 6.0. In: *Lecture Notes in Computer Science*, vol. 1786, Springer-Verlag, pp. 354–357.
- Lindemann, C., 1995. DSPNexpress: a software package for the efficient solution of deterministic and stochastic Petri nets. *Performance Evaluation* 22 (1–2), 3–21.
- Mahevas, S., Rubino, G., 2001. Bound computation of dependability and performance measures. *IEEE Transactions on Computers* 50 (5), 399–413.
- Nicola, V.F., Shahabuddin, P., Nakayama, M.K., 2001. Techniques for fast simulation of models of highly dependable systems. *IEEE Transactions on Reliability* 50 (3), 246–264.
- Plateau, B., Stewart, W.J., Silva, M., 1999. Numerical solution of Markov chains. In: *Proceedings of the 3rd International Conference on Numerical solution of Markov chains*, Prensas Universitarias de Zaragoza, Spain, pp. 435–448.
- Qureshi, M.A., Sanders, W.H., Van Moorsel, A.P.A., German, R., 1996. Algorithms for the generation of state-level representations of stochastic activity networks with general reward structures. *IEEE transactions on Software Engineering* 22 (9), 603–614.
- Ramani, S., Trivedi, K.S., Dasarathy, B., 2000. Performance analysis of the CORBA event service using stochastic reward nets. In: *Proceedings of the 19th Symposium on Reliable Distributed Systems*, Nurnberg, Germany, pp. 238–247.
- Stewart, W.J., 1994. *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press.

Samir M. Koriem received his B.Sc. in electronics and communication engineering from the University of Helwan, Cairo, Egypt, in 1978. In 1979, he joined the department of systems and computers engineering, faculty of engineering, university of Al-Azhar, Cairo, as a demonstrator. In 1985, he received his M.Sc. in computer engineering from Al-Azhar University. From 1985 to 1988, he was an assistant lecturer in the systems and computers engineering department, university of Al-Azhar. In 1993, he received his Ph.D. in computer engineering from the computer science and automation department of the Indian institute of science, Bangalore, India. From 1994 to 1999, he was an *assistant professor* in the systems and computers engineering department, university of Al-Azhar. From 1999 till now, he is an *associate professor* in the systems and computers engineering department, university of Al-Azhar. His research interests are in the areas of modeling and performance evaluation of parallel and distributed computing systems, real-time systems, multimedia systems, computer networks, performance analysis, and modeling of system reliability, and analysis of time, color, fuzzy, neural, and stochastic Petri nets. He published several papers in international conferences and journals such as the journal of systems and software (USA), the computer journal (UK), computer communication (UK), parallel processing conference (USA), and IEICE conference (Japan). He also reviews in several international conferences and journals. He is a member of the COMNET CLUB (Netherlands) that concerned with the advanced publications in the area of computer networks.